

# Market-Place-Inc (MPI) – TP3 Sistemas Distribuidos

## Respuestas a las consignas teóricas:

### Pregunta 1 — Sobre el problema de la concurrencia y el impacto del Overselling

El término **overselling** (o sobreventa) en el contexto de la ingeniería de software y sistemas distribuidos refiere a un estado de inconsistencia de datos provocado por una **condición de carrera (race condition)**. Este fenómeno ocurre cuando múltiples hilos de ejecución o instancias de un servicio intentan leer y modificar un recurso compartido de forma simultánea sin los mecanismos de aislamiento y exclusión mutua adecuados.

Para ilustrar este problema con un escenario concreto, supongamos que nuestro inventario registra que queda exactamente **1 iPhone** disponible en stock. En el mismo segundo, dos usuarios distribuidos geográficamente, María y Carlos, presionan el botón de compra. Dado que las peticiones se procesan en paralelo por distintas instancias de nuestro backend de FastAPI, ambas lecturas a la base de datos se ejecutan de manera casi idéntica antes de que impacte cualquier escritura. La instancia que atiende a María lee que el stock es 1; simultáneamente, la instancia que atiende a Carlos lee exactamente lo mismo: stock es 1. Al no existir restricciones, el sistema valida positivamente ambas transacciones. María concreta la operación y el sistema decrementa el stock a 0. Acto seguido, la transacción de Carlos finaliza y resta una unidad a un stock que ya era 0, dejando el registro en **-1**.

```
Instancia A (María) ---- [Lee Stock: 1] -----> [Resta 1  
-> Stock: 0] (Ok)  
Instancia B (Carlos) ----- [Lee Stock: 1] ----->  
[Resta 1 -> Stock: -1] (Overselling)
```

Para Market-Place-Inc (MPI), este desfasaje técnico se tradujo en una pérdida real de **\$2.000.000 de dólares** durante el Hot Sale pasado debido a los costos operativos ocultos de la falla distribuida:

- **Costos financieros directos:** Las pasarelas de pago cobran comisiones fijas y variables por cada transacción y por cada proceso de reversión o reembolso (*refund*). Devolver el dinero de miles de compras inválidas implica perder esas comisiones.
- **Saturación del soporte técnico:** El volumen de reclamos simultáneos obliga a la empresa a pagar horas extras al personal de atención al cliente o contratar infraestructura adicional para los centros de ayuda.
- **Penalizaciones y pérdida de reputación:** En plataformas de e-commerce de alta competencia, vender sin stock degrada los algoritmos de posicionamiento de la tienda y, en el peor de los casos, acarrea severas multas legales por publicidad engañosa o incumplimiento de contrato.

Si este escenario se multiplica por 1000 veces en un evento masivo como el Hot Sale, el impacto no es lineal, sino catastrófico. No solo se destruye el flujo de caja por la

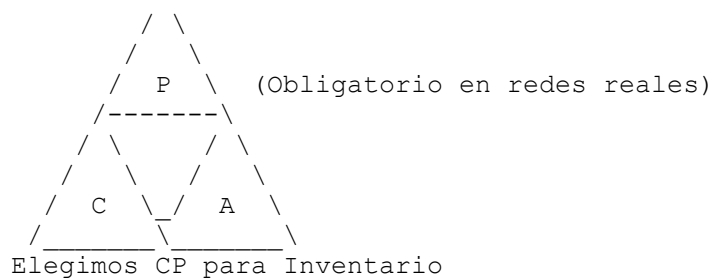
logística inversa de reembolsos, sino que las bases de datos colapsan bajo la carga de escrituras que intentan enmendar los registros negativos, arrastrando a todo el ecosistema de microservicios hacia una falla sistémica generalizada.

## Pregunta 2 — Análisis del Teorema CAP aplicado al módulo de Inventario

El **Teorema CAP** (atribuido a Eric Brewer) es uno de los pilares fundamentales del diseño arquitectónico distribuido. Establece que en cualquier sistema de datos compartido que sufra una partición de red, es matemáticamente imposible garantizar simultáneamente más de dos de las siguientes tres propiedades:

- **Consistencia (Consistency - C):** Implica que todos los nodos del sistema distribuidos ven exactamente los mismos datos al mismo tiempo. Si realizamos una escritura en el nodo A, cualquier lectura posterior en el nodo B debe reflejar esa actualización de inmediato.
- **Disponibilidad (Availability - A):** Garantiza que cada nodo que no haya fallado devuelva siempre una respuesta (que no sea un error), sin importar si contiene el dato más reciente o no.
- **Tolerancia a Particiones (Partition Tolerance - P):** Es la capacidad del sistema de seguir funcionando a pesar de que la red sufra caídas, demoras o divisiones que impidan la comunicación directa entre sus nodos.

Dado que en el mundo real las redes son inherentemente poco confiables, **la tolerancia a particiones (P) no es opcional**; debemos asumirla como una realidad de los sistemas distribuidos. Por lo tanto, el dilema arquitectónico real se reduce a elegir entre **Consistencia y Tolerancia a Particiones (C+P)** o **Disponibilidad y Tolerancia a Particiones (A+P)**.



Para el módulo de inventario de MPI, nuestro equipo ha seleccionado deliberadamente una estrategia **C+P (sacrificando la disponibilidad)**. Esto significa que si un nodo que maneja el stock de iPhones pierde conectividad con el nodo líder o el clúster central debido a una partición de red, el sistema preferirá denegar el servicio o responder con un error estructurado ("No se puede verificar el stock en este momento") antes que arriesgarse a entregar un dato desactualizado.

Si hubiésemos optado por un enfoque **A+P**, el sistema priorizaría responder siempre "Sí, hay stock" para mantener la disponibilidad de la venta alta. Sin embargo, bajo una partición de red, las distintas instancias de la base de datos no podrían sincronizar sus decrementos, desencadenando un escenario masivo de *overselling*. En el dominio de los negocios digitales, el costo económico y reputacional de una venta rechazada temporalmente (enfoque CP) es sustancialmente menor que el costo logístico, legal y

operativo de vender un producto inexistente (enfoque AP). El inventario requiere consistencia fuerte; el dinero real de la empresa depende de la precisión de ese registro.

### Pregunta 3 — El Algoritmo de Consenso Raft y la importancia del Quórum

El algoritmo de consenso **Raft** fue diseñado para gestionar la replicación de logs en sistemas distribuidos, resolviendo el problema de cómo un conjunto de servidores independientes puede ponerse de acuerdo para mantener un estado único y coherente, incluso ante fallas de red o caídas de nodos. Para lograrlo, Raft descompone el problema en subproblemas, siendo el principal la **Elección de Líder**.

En Raft, un servidor solo puede encontrarse en uno de estos tres estados:

- **Leader (Líder):** Es el único nodo encargado de recibir las peticiones de los clientes, gestionar la replicación de los logs y enviar notificaciones periódicas (*heartbeats*) a los demás para mantener su autoridad.
- **Follower (Seguidor):** Es un estado completamente pasivo. No toma decisiones por sí mismo; solo responde a los llamados de procedimiento remoto (RPC) que provienen de los líderes o candidatos.
- **Candidate (Candidato):** Es un estado de transición que asume un seguidor cuando deja de recibir los *heartbeats* del líder actual dentro de un tiempo límite (*election timeout*), iniciando formalmente un nuevo proceso de votación para convertirse en líder.

Para entender la mecánica del consenso, analicemos un clúster compuesto por **5 servidores** ( $N=5$ ). Si el líder actual sufre una caída total, los seguidores detectan la ausencia de actividad al vencerse sus temporizadores. El primer nodo en despertar pasa al estado de **Candidate**, incrementa el término de la elección y envía una petición de voto al resto de los 4 nodos vivos.

```
[Server 1] (Candidato) -- Solicita votos --> [Server 2], [Server 3],  
[Server 4], [Server 5]
```

Para que este candidato se corone como el nuevo líder legítimo del sistema, Raft exige estrictamente obtener la **mayoría absoluta** de los votos de la configuración total del clúster, conocida formalmente como **quórum**. La fórmula matemática estándar para calcular el quórum es:

$$\text{Quórum} = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

Para un grupo de 5 servidores, el quórum requerido es indefectiblemente de **3 votos** (el propio voto del candidato más 2 votos de nodos seguidores).

Esta necesidad de mayoría absoluta es crítica para evitar el catastrófico fenómeno del **Split-Brain (cerebro dividido)**. Si el algoritmo permitiese elegir un líder con solo 2 votos en un clúster de 5, una partición de red podría dividir el sistema en dos subgrupos aislados: uno de 2 nodos y otro de 3 nodos. Si el subgrupo de 2 nodos pudiera elegir su propio líder (al requerir solo 2 votos) y el subgrupo de 3 nodos eligiera de forma independiente el suyo, tendríamos **dos líderes activos simultáneamente** escribiendo

datos contradictorios en el mismo sistema. Al requerir obligatoriamente 3 votos, el subgrupo de 2 nodos jamás podrá alcanzar el quórum, garantizando que solo exista un único líder válido en todo el sistema distribuido a la vez.

## Pregunta 4 — Anatomía y Seguridad de un Bloqueo Distribuido con Redis

Para implementar nuestro candado distribuido a nivel de aplicación, utilizaremos Redis como un almacén de estructuras de datos en memoria compartido. El comando propuesto para inicializar el bloqueo es el siguiente:

Python

```
redis.set("lock:iphone", "user_a", nx=True, ex=5)
```

Analizando la instrucción línea por línea y parámetro por parámetro desde la perspectiva de la API de Redis, comprendemos el siguiente comportamiento:

- `redis.set(...)`: Invoca el comando elemental `SET`, cuyo propósito base es almacenar el valor provisto en una clave específica dentro del espacio de nombres de Redis.
- `"lock:iphone"`: Es el identificador único o `Key` que representa el recurso crítico que deseamos proteger. Actúa conceptualmente como el "candado físico" del stock de iPhones.
- `"user_a"`: Es el `Value` asociado a la clave. No debe ser un string genérico; tiene que ser un identificador único global (como un UUID o el ID de la sesión) correspondiente al proceso o cliente que está solicitando el bloqueo. Esto nos permitirá validar posteriormente que solo el legítimo dueño del candado pueda liberarlo, evitando que un proceso B borre por error el candado del proceso A.
- `nx=True`: Habilita la bandera condicional **NX (Set if Not Exists)**. Esta opción le indica a Redis que la operación de escritura solo debe completarse de forma exitosa si la clave `"lock:iphone"` **no existe previamente** en la base de datos. Si la clave ya existe, Redis aborta la operación devolviendo un valor nulo (`None` o `False`). Este parámetro es fundamental porque garantiza la propiedad de **exclusión mutua** de manera atómica: solo el primer proceso en llegar logrará escribir la clave.
- `ex=5`: Establece un tiempo de expiración o **TTL (Time-To-Live)** expresado en segundos. Transcurrido ese lapso de tiempo exacto, Redis eliminará la clave de forma automática e interna del sistema sin necesidad de comandos externos.

```
[FastAPI Instancia 1] -- SET lock:iphone user_a NX EX 5 --> [ Redis ]  
(Éxito: Clave creada 🔒)  
[FastAPI Instancia 2] -- SET lock:iphone user_b NX EX 5 --> [ Redis ]  
(Falla: Clave ya existe ❌)
```

Las consecuencias de omitir o configurar incorrectamente estos parámetros alterarían por completo la estabilidad de nuestra arquitectura:

Si **NO** usáramos el parámetro `nx=True`, el comando `SET` se comportaría como una escritura tradicional sobrescribiendo ciegamente cualquier valor previo. En la práctica, esto rompería el concepto de candado: el proceso B pisaría el registro del proceso A,

ambos creerían tener acceso exclusivo al inventario simultáneamente y el sistema volvería a sufrir de *overselling*.

Si **NO** configuráramos el tiempo de expiración con `ex=5` y dependiéramos únicamente de que nuestra aplicación borre el candado manualmente al finalizar la transacción (`redis.delete()`), nos expondríamos a un escenario severo de **Deadlock (bloqueo mutuo)**. Si el contenedor de FastAPI que adquirió el candado sufre un fallo crítico de hardware, un error de memoria fuera de rango (*Out Of Memory*), o experimenta una pausa prolongada por el Garbage Collector justo después de obtener el candado pero antes de decrementar el stock, la clave `"lock:iphone"` quedaría alojada en Redis de manera indefinida. Como consecuencia, ningún otro usuario en el mundo podría volver a intentar comprar un iPhone, congelando el proceso de negocio por completo. El TTL es nuestra red de seguridad distribuida.

## Pregunta 5 — Observabilidad, Monitoreo Específico y Mitigación de Fallas

En las arquitecturas modernas de microservicios, la infraestructura se vuelve altamente dinámica y compleja, lo que transforma a la **observabilidad** en un requisito no funcional crítico. Para dar respuesta a esto incorporamos herramientas complementarias como **Prometheus** y **Grafana**:

- **Prometheus** opera como nuestro motor de telemetría y base de datos de series temporales. Utiliza un modelo basado en *pull*, lo que significa que realiza consultas HTTP periódicas (*scraping*) a endpoints específicos de nuestros microservicios para recolectar métricas numéricas crudas sobre el estado del hardware, los tiempos de respuesta y la lógica de negocio.
- **Grafana** se posiciona como la capa de visualización e inteligencia operacional. Consume las series temporales expuestas por Prometheus y las transforma en tableros visuales (*dashboards*), gráficos de líneas, mapas de calor y sistemas de alertas en tiempo real que son fácilmente interpretables por el equipo de ingeniería.

```
[ Microservicios FastAPI ]
    ^
    | (Scraping HTTP /metrics)
[ Prometheus (Métricas) ] ---> [ Grafana (Dashboards / Alertas) ]
```

Poner en producción un sistema distribuido de la magnitud de MPI sin una suite de monitoreo adecuada equivale a **operar un avión a ciegas**. Ante una eventual falla de performance, el equipo de desarrollo carecería por completo de visibilidad interna para diagnosticar el problema de raíz. Sin monitoreo, la única forma de enterarse de que el sistema se está rompiendo es a través de los **síntomas externos de los clientes**: cuando las redes sociales se inundan de quejas, los canales de soporte colapsan o, peor aún, cuando los reportes contables del final del día muestran pérdidas millonarias debido a transacciones caídas. Estaríamos en una postura puramente reactiva, obligados a leer archivos de log inconexos en múltiples servidores en plena crisis.

Para asegurar de forma proactiva la salud del mecanismo de candados distribuidos que estamos diseñando para el e-commerce, una de las métricas más críticas que debemos instrumentar y medir mediante Prometheus es:

`lock_acquisition_latency_seconds`

Esta métrica de tipo *Histogram* mide el intervalo de tiempo exacto que le toma a nuestras instancias de FastAPI conectarse a Redis, ejecutar la primitiva atómica y obtener una confirmación exitosa o de rechazo del candado.

**¿Por qué es importante medirla?** Si observamos en Grafana que el percentil 99 (p99) de esta métrica se eleva repentinamente por encima de los 500 milisegundos, sabremos de inmediato que Redis está experimentando una degradación severa (por ejemplo, saturación de la CPU por comandos mal optimizados o problemas de latencia en la red subyacente). Detectar esta anomalía a tiempo a través de una alerta automatizada nos permite escalar la infraestructura de Redis o activar mecanismos de contingencia (*circuit breakers*) mucho antes de que el retraso se propague en cascada hacia el servicio de Pedidos y termine por tumbar la interfaz del Frontend frente al usuario.

# Dashboards de Graphana

